

GRAPH PLATFORM 3.0.0

Graph Platform v3

Product & Technical Handbook

Knowledge graphs. Workflows. RAG. Local AI.

Public read-only domains + protected workspaces.

Redesigned frontend and Admin Console.

Документ описывает архитектуру, UX, API, модель доступа,
offline AI, данные, сценарии, эксплуатацию и ограничения проекта.

Generated from the improved project source | July 2026

0. Паспорт версии

Graph Platform v3 в одной фразе

Self-hosted knowledge operating system, где доменные знания, связи, участники, процессы, документы и AI-контекст живут в одном графе, а опубликованные домены можно безопасно читать без регистрации.

Параметр	Значение
Версия	3.0.0
Frontend	React 18 + Vite + TypeScript + @xyflow/react + Zustand
Backend	Node.js / Express 4, better-sqlite3, JWT, bcryptjs
Offline AI	Python standard library, Hybrid Offline AI v3
Хранилище	SQLite WAL; graph/workspace scoped schema из 20 таблиц
Базовые домены	Bank и Law, public read-only
Внешний LLM	Опциональный OpenAI-compatible gateway: DeepSeek/OpenAI и совместимые провайдеры
Deployment	Same-origin frontend + /api reverse proxy; Offline AI в private/loopback сети

Что изменилось относительно демо-состояния

Frontend перепозиционирован из технического демо в продуктовый dashboard: hero/value proposition, каталог доменов, control center, явный health/status, сильнее визуальная иерархия и AI-подача.

Admin Panel превращён в Admin Console с Overview, Domains, Knowledge Package, Users, AI Ops и System.

Public/private стали частью модели графа: guest читает опубликованные домены, но любые изменения требуют прав.

Регистрация больше не даёт права записи в общий demo-workspace: создаётся персональный workspace.

Backend сведён к одной SQLite connection layer; graph_id проходит через данные, RAG, reviews, bindings и AI history.

Offline AI больше не дописывает случайный char-RNN текст: используется локальный retrieval, graph-context, memory, confidence и sources.

Knowledge Package импорт стал collision-safe: чужие node/edge IDs ремар-ятся вместо перезаписи другого домена.

Workspace Library дополнена недостающими endpoints для Actors, templates и независимого clone проекта.

1. Продуктовая концепция

Платформа отвечает на проблему разрыва между документацией, бизнес-процессами, архитектурой, ролями и AI. Вместо набора несвязанных страниц она представляет организационное знание как живой graph. Node может быть доменом, процессом, системой, артефактом, нормативным объектом или другой сущностью; Edge фиксирует смысловую связь; Actor и Role Bindings показывают ответственность; Work Items связывают знание с исполнением.

Ключевая продуктовая петля

1. Publish domain -> public knowledge surface
2. Explore graph -> select node/layer/tab/role
3. Ask Copilot -> graph + work items + RAG become context
4. Work in private workspace -> projects, actors, reviews, templates
5. Govern in Admin Console -> visibility, users, imports, AI history
6. Export / evolve Knowledge Package -> graph becomes reusable asset

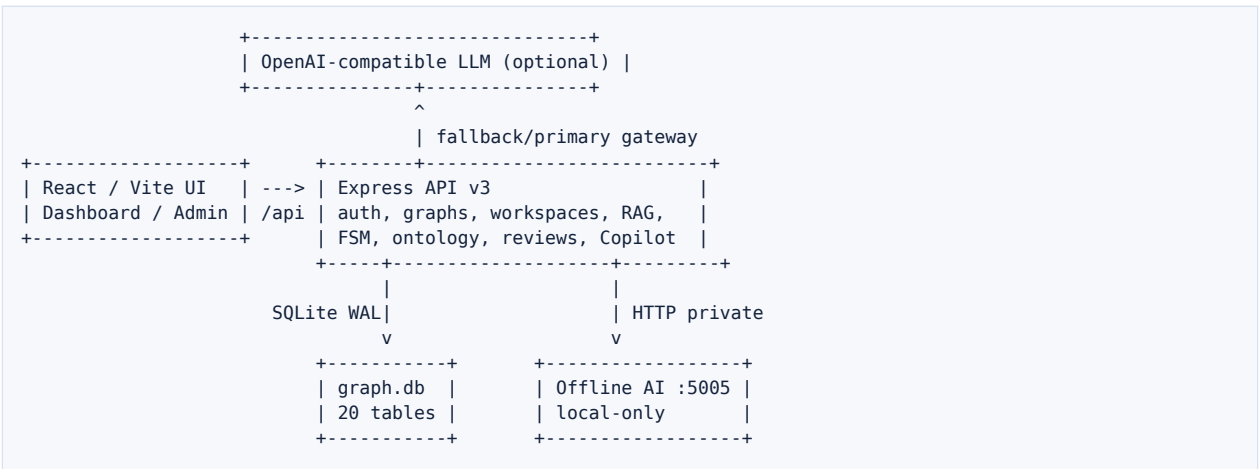
Три состояния графа

Tab	Смысл	Типичный вопрос
As is	Фиксация текущей реальности: системы, роли, знания и разрывы	Что существует сейчас и где проблема?
Process	Путь трансформации, пилоты, перенос знаний, переходные этапы	Как перейти от текущего к целевому?
To be	Целевая knowledge operating model	Как должна работать система после изменений?

Почему frontend - важная часть продукта

Главный экран теперь не заставляет пользователя угадывать структуру. Сначала он видит ценность платформы и доступные домены, затем выбирает граф и получает единый control center. Это делает продукт продаваемым не только инженеру, но и владельцу домена, архитектору, юристу, аналитику или руководителю.

2. Архитектура системы



Runtime boundary

Browser никогда не получает `OFFLINE\_AI\_KEY`; Copilot вызывается через Backend.

Backend является policy boundary: определяет public/private, membership, role, graph scope и mutation rights.

Offline AI получает уже собранный Backend контекст, поэтому локальный fallback видит тот же активный graph/RAG context, а не только текст вопроса.

SQLite работает через единственный singleton connection, WAL, foreign_keys и busy_timeout; soft migrations поддерживают старые demo базы.

Структура репозитория

frontend/	React/Vite product UI
src/App.tsx	dashboard, routing, graph control center
src/components/	FlowCanvas, ChatSidePanel, AdminPage, Library...
backend/	Express API + SQLite
src/routes/	auth, public, graphs, graph, rag, copilot, admin...
src/db/	schema, migrations, seed
src/engines/	ontology, FSM, LLM gateway
offline-ai/	zero-dependency local AI HTTP service
Data/	offline knowledge corpus
test/	zero-dependency API smoke test
docs/	supporting project reports
info.pdf	this handbook

3. Frontend v3 - продаваемый product surface

Главный dashboard

Hero объясняет продукт через outcome: знания, процессы и AI в одном живом графе.

Domain catalog визуально разделяет опубликованные области знаний; public-графы доступны до логина.

Graph control center объединяет выбор домена, tabs As is/Process/To be, layers и role projection.

Graph stage остаётся центральным visual object, но окружён ясным продуктовым контекстом, а не набором технических controls.

Health/status, profile/admin navigation и workspace state видны и восстанавливаются после reload.

Frontend создаёт `gp\_session` и отправляет `X-Session-Id`, чтобы Copilot history не смешивалась между browser sessions.

Graph visualization

`FlowCanvas` использует `@xyflow/react`. UI поддерживает выделение узлов, inspector, path finder, layer filtering и presentation mode. Backend отдаёт graph-scoped nodes/edges, поэтому визуализация не должна случайно смешивать два домена при выбранном `X-Graph-Id`.

AI UX

`ChatSidePanel` показывает не только текст ответа, но и model/confidence metadata, suggestions и более чистый conversational layout. Внешний LLM не является обязательным: при его отсутствии UI продолжает работать через локальный hybrid engine.

Login и profile

Login screen больше не публикует встроенный admin пароль как часть UI. После reload token проверяется через `/api/auth/me`, затем восстанавливаются user/workspace states. Public catalog остаётся полезным даже без аккаунта, поэтому login - это переход к приватным mutation/workspace возможностям, а не gate перед всей системой.

Runtime config

```
window.__GP_CONFIG__ = {
  API_URL: "", // same-origin /api recommended
  OFFLINE_AI_URL: "http://127.0.0.1:5005",
  APP_TITLE: "Graph Platform"
};
```

4. Admin Console

Admin Console проектировалась как операционная поверхность платформы, а не как набор CRUD-кнопок.

Раздел	Что даёт
Overview	System summary: users, workspaces, graphs, nodes, edges, documents, questions, public graphs, recent AI activity, rate-limit state.
Domains	Create graph, visibility public/private, export/delete, быстрый контроль каталога.
Knowledge	Import Knowledge Package; auto-create graph; replace/scoped import; collision-safe remap.
Users	Список пользователей, смена global role member/admin, удаление с защитой от self-delete.
AI Ops	История вопросов/ответов и model metadata для анализа использования Copilot.
System	Health/version/LLM/public-domain mode и operational status.

Import integrity

- Если `graphId` отсутствует, backend создаёт новый graph и slug.
- Node IDs, уже занятые в другом graph, получают новый UUID; edges автоматически используют remapped endpoints.
- Edge IDs тоже remap-ятся при collision.
- Edge отклоняется, если source/target не принадлежат целевому graph.
- Replace работает scoped к выбранному graph/workspace, а не как глобальное стирание данных.

5. Public/private access model

Главное правило

Публикация домена открывает только чтение знаний. Права на изменение никогда не выводятся из `visibility=public`; для mutation backend проверяет authenticated workspace access.

Субъект	Public read	Private read	Workspace write	Admin ops
Guest	Да	Нет	Нет	Нет
Member	Да	Только membership	Только membership	Нет
Global admin	Да	Да	Да	Да
Service API key	Да	Да	Да	Да

Unauthenticated domain endpoints

```
GET /api/public/domains
GET /api/public/domains/:slug
GET /api/public/domains/:slug/graph?tab=tobe
GET /api/graphs           # guest sees public graphs only
POST /api/copilot/chat    # guest may ask within a public graph
```

Почему не открыт write

Публичный knowledge catalog часто нужен для витрины, документации, customer portal или публичного Copilot. Но публикация не должна давать возможность менять ontology, импортировать JSON, добавлять RAG, создавать edges или управлять users. Поэтому read path и mutation path намеренно разделены.

Signup isolation

Self-registration по умолчанию создаёт `ws-` с Default First ontology и membership role `admin` внутри собственного workspace, при этом global user role остаётся `member`. Автоматическое вступление в `ws-default` отключено. При необходимости shared onboarding разрешается только через явный `PUBLIC\_REGISTRATION\_WORKSPACE\_ID`.

6. Backend API v3

Surface	Endpoints	Access
Health	GET /api/health	Public
Public domains	GET /api/public/domains; /:slug; /:slug/graph	Public read-only
Auth	POST /api/auth/register; /login; GET /auth/me	Register/login public; me JWT
Graphs	GET/POST /api/graphs; PATCH/DELETE /api/graphs/:id	GET mixed policy; mutations protected
Graph data	GET /api/graph/nodes; /graph/edges; /actors; /work-items; /interest-scope/:actorId	Public selected graph or workspace member
FSM	GET machines/transitions; POST transition	Read policy + protected transition
Ontology	GET /api/ontology; POST /ontology/extend	Public selected graph read; extend protected
RAG	GET documents/search; POST ingest	JWT + workspace/graph checks
Copilot	POST /api/copilot/chat; GET /copilot/history	Public graph chat; history protected
Reviews	GET/POST /api/reviews	Public graph read; write protected
Bindings	GET/POST/DELETE /api/role-bindings	Public selected graph read; write protected
Workspace	GET/POST /workspaces; actors; projects; templates	JWT membership
Admin	/api/admin/*	JWT admin/service
Ratings	GET/POST /api/ratings	Default product feed public; other workspace protected

Request context headers

Header	Назначение
Authorization	Bearer JWT пользователя
X-API-Key	Service access для server-to-server операций
X-Workspace-Id	Выбор workspace; guest header игнорируется, membership проверяется
X-Graph-Id	Активный domain graph для data/RAG/Copilot scope
X-Session-Id	Диалоговая изоляция Copilot

7. Data model

Schema содержит 20 таблиц. Основная идея - workspace задаёт tenant boundary, graph задаёт domain boundary, а рабочие сущности при возможности несут `graph\_id`.

Группа	Таблицы	Роль
Tenant/Auth	workspaces, users, memberships	Identity и workspace membership
Domain	graphs, ontology	Публичность, metadata, domain ontology
Knowledge	nodes, edges	Семантический граф
Delivery	portfolios, projects, work_items, sprints, pipes	Исполнение и трансформация
People	actors, role_bindings	Участники и контекстные роли
Governance	reviews, ratings	Обратная связь и review flow
RAG	documents, chunks	Документы и retrieval chunks
AI	questions	История ответа, graph/session context и RAG refs
Reuse	templates	Замороженные project snapshots

Ключевые invariants

- Graph belongs to one workspace.
- Public visibility разрешает read, но не меняет membership.
- Graph-scoped edge должен ссылаться на nodes этого graph.
- RAG document/chunk сохраняет workspace_id + graph_id.
- Question сохраняет workspace_id + graph_id + session_id, поэтому диалог можно изолировать и анализировать.
- Template snapshot копирует данные независимо: node IDs ретар-ятся при создании нового проекта.

SQLite runtime

`database.js` создаёт одну connection, включает WAL, foreign_keys и busy_timeout. На startup применяется schema и безопасные additive migrations. Для старых демо данных есть ограниченный backfill известных Bank/Law IDs; произвольные customer rows не угадываются и не переносятся между доменами автоматически.

8. Graph Copilot и Offline AI

LLM gateway

`engines/llm.js` сначала использует OpenAI-compatible provider, если задан ключ. В контекст входят выбранный graph, nodes, edges, work items и top RAG chunks. Если внешний provider отсутствует или недоступен, тот же context отправляется в локальный Offline AI. Если и локальный HTTP service недоступен, Backend имеет последний deterministic local fallback.

Offline pipeline

```
question
-> tokenize + Russian light stemming
-> synonym expansion + short-dialog memory
-> BM25 over Data/*.txt/*.md
-> character trigram fuzzy scoring
-> graph-context scoring
-> merge + sentence reranking + dedup
-> intent answer + grounded extractive synthesis
-> confidence + sources + retrieval stats
```

Что делает AI умнее старой версии

Контекст выбранного graph поступает в offline fallback так же, как во внешний LLM.

BM25 учитывает частоту терминов и редкость слов, а не только простое совпадение TF-IDF строк.

Char trigrams помогают при неточных формулировках, а light stemming - при русских падежах/окончаниях.

Synonym groups связывают graph/domain/actor/RAG/FSM/security/legal термины.

Session memory используется для коротких follow-up вопросов; анонимные clients без session ID не получают общую историю.

Ответ содержит sources/confidence и не должен придумывать факт, если локальный corpus/context недостаточен.

Случайная char-level RNN генерация удалена как источник недостоверного текста.

Privacy

Offline AI запускается на `127.0.0.1:5005` по умолчанию, использует `X-Offline-Key` и не требует внешних Python libraries. Он может полностью работать без интернета. При использовании внешнего LLM context покидает локальную машину согласно политике выбранного провайдера - это отдельное deployment решение.

Корпус

`Data/` содержит архитектуру, слои, actors, work items, review/execution, reporting, security/RBAC, RAG, ontology, transformation, admin, public access v3 и legal domain. Runtime `/reload` перечитывает corpus без переобучения нейросети.

9. Workspace Library и reusable delivery

Workspace Library связывает knowledge model с реальной работой. Проект может иметь actors через role bindings, быть заморожен в template и затем клонирован как независимый новый project.

Template flow

```
Project
-> snapshot selected project's nodes + internal edges + work items + ontology
-> Template v1
-> POST /api/projects { templateId }
-> create new Project
-> remap every node id
-> recreate only internal edges with new endpoints
-> recreate work items with remapped relatedNodeIds
```

Candidate actors для `unassigned\_to=` дополнительно фильтруются по graph проекта, поэтому actor другого domain не предлагается случайно. Backend всё равно повторно валидирует graph/workspace при записи binding.

10. Security model

Контроль	Реализация
Password storage	bcryptjs; cost 12 production, 10 development
JWT	HS256, configurable expiry (default 24h), production secret required
Service auth	X-API-Key; production value required
Tenant isolation	memberships + workspace validation
Domain isolation	graph visibility + graph.workspace_id + graph-scoped queries
Public surface	read-only published domains
Mutation	authRequired + membership/admin checks
Input limits	Express JSON size, chat validation, offline body/message limits
Rate limiting	endpoint-aware in-memory rate limiter
Headers	security headers, no powered-by, no-store where appropriate
Offline AI	loopback default + service key + max body

Production requirements

Use HTTPS at reverse proxy. README demo HTTP URLs are for local development only.

Set strong JWT_SECRET, API_KEY, OFFLINE_AI_KEY and ADMIN_INITIAL_PASSWORD before first production bootstrap.

Restrict CORS to trusted origins and set TRUST_PROXY correctly behind a proxy.

Do not expose port 5005 publicly; keep Offline AI on loopback/private network.

Back up SQLite correctly with WAL awareness and verify restore procedures.

For high write concurrency or multi-node deployment, migrate repository layer to PostgreSQL; HoneyORM file is an archived concept, not an active production adapter.

Security boundary

Нельзя считать frontend проверку прав security-механизмом. Все критические решения выполняются Backend: visibility, workspace membership, graph ownership, admin role и cross-domain integrity.

11. Knowledge Package format

Knowledge Package нужен для обмена domain model с AI/другими системами, резервного переноса или создания нового домена из JSON.

```
{
  "name": "My Domain",
  "slug": "my-domain",
  "description": "...",
  "visibility": "private",
  "ontology": { ... },
  "nodes": [
    { "id": "root", "label": "Root", "layer": "Knowledge", "nodeKind": "domain" }
  ],
  "edges": [
    { "id": "e1", "source": "root", "target": "child", "label": "CONTAINS" }
  ]
}
```

Import endpoint проверяет workspace, graph ownership, duplicate IDs внутри пакета и endpoints edges. Collision с ID другого graph не уничтожает чужие данные: создаётся новый UUID и mapping применяется к edge endpoints.

12. Запуск и эксплуатация

Development

```
# Backend
cd backend
cp .env.example .env
npm ci
npm run dev

# Offline AI
cd offline-ai
python3 server.py

# Frontend
cd frontend
npm ci
npm run dev -- --host 0.0.0.0
```

Seed

На пустой development DB создаются Bank и Law, demo workspace и локальный admin `admin@graph.local`. Development bootstrap password - `Admin1234!`. В production seed требует `ADMIN\_INITIAL\_PASSWORD`, поэтому этот демо пароль не становится production default.

Environment

Переменная	Назначение
PORT	Backend port, default 3001
NODE_ENV	development / production
JWT_SECRET	JWT signing secret; mandatory production
JWT_EXPIRY	Token lifetime
API_KEY	Service API key; mandatory production
ADMIN_INITIAL_PASSWORD	Initial admin bootstrap on empty production DB
PUBLIC_REGISTRATION_WORKSPACE_ID	Optional explicit shared signup workspace
CORS_ORIGINS	Allowed browser origins
TRUST_PROXY	Reverse proxy behavior
SQLITE_PATH	Custom graph.db path
OPENAI_API_KEY / DEEPSEEK_API_KEY	Optional external LLM
OPENAI_BASE_URL / MODEL	OpenAI-compatible provider config
OFFLINE_AI_URL / KEY	Local AI service URL and secret

Recommended production topology

```
Internet
-> HTTPS reverse proxy
  -> static frontend/dist
  -> /api -> Backend :3001
      -> SQLite volume
      -> Offline AI 127.0.0.1:5005
      -> External LLM only when policy allows
```

13. Проверки, выполненные при доработке

Проверка	Результат
Backend JavaScript syntax	Все active backend/src/*.js проходят `node --check`.
Frontend TypeScript	`tsc --noEmit` проходит после redesign и session patch.
Offline AI syntax	`python3 -m py_compile` проходит.
Offline HTTP	Health=200; bad key=401; chat возвращает hybrid model, confidence, sources.
Guest catalog	Bank/Law и другие public graphs читаются без token.
Guest mutation	POST graph отклоняется 401.
Signup isolation	Новый user получает personal workspace, public graphs canEdit=false.
Cross-workspace write	Попытка patch Bank новым member отклоняется 403.
RAG	Ingest/search сохраняют active graph_id.
Template clone	Snapshot копирует nodes/edges независимо с remap IDs.
Import collisions	Node/edge collisions remap; исходный Bank node не перезаписывается.
Cross-domain edge	Admin edge между узлами разных graphs отклоняется 400.
Copilot E2E	Backend -> Offline AI получает Bank graph/RAG context и session ID.
API smoke	Standard-library test/smoke.py проходит полный основной сценарий.

Build environment caveat

Production Vite build в текущем sandbox не был повторно завершён не из-за TypeScript/source error, а из-за платформенного `node\_modules` исходного ZIP: в нём отсутствует Linux optional package `@rollup/rollup-linux-x64-gnu`. Попытка clean dependency install упиралась во временно недоступный package registry. TypeScript check проходит. На нормальной Linux/CI машине после `npm ci` Vite должен получить корректный Rollup binary; это следует подтвердить в вашем CI перед релизом.

14. Ограничения и roadmap

V3 значительно взрослее исходного проекта, но несколько границ важно понимать честно.

Сейчас	Что делать при росте
SQLite single-node	Для multi-node/high-write нагрузки перейти на PostgreSQL repository adapter.
In-memory rate limiter	В cluster deployment вынести counters в Redis/gateway/WAF.
Offline AI retrieval/extractive	Для сложного generative reasoning использовать локальную LLM или OpenAI-compatible provider с policy controls.
Global admin role	В enterprise варианте разделить platform admin и tenant admin permissions.
Knowledge Package JSON	Добавить formal JSON Schema/version migration/signing для межсистемного обмена.
Operational telemetry	Добавить structured logs, metrics, traces, alerting, audit log.
SQLite migrations	Перевести soft additive migrations в versioned migration runner перед крупной схемой.

Roadmap приоритет

P1: CI pipeline: clean npm ci, typecheck, Vite build, backend smoke, security tests.

P1: formal audit log для admin/mutations и export/import events.

P1: PostgreSQL adapter + versioned migrations для production scale-out.

P2: richer local embeddings/LLM, если deployment допускает локальную модель большего размера.

P2: tenant admin roles, invitations, password reset/email verification.

P2: signed/versioned Knowledge Package schema и validation report в Admin Console.

Итог

Graph Platform v3 уже имеет цельную продуктовую модель: public knowledge surface, private workspaces, graph-aware backend, reusable projects/templates, Admin Console и локальный AI. Следующий уровень зрелости - CI, auditability, PostgreSQL/Redis scale-out и enterprise identity lifecycle.

15. Краткая карта файлов

Файл/директория	Зачем нужен
frontend/src/App.tsx	Главный product dashboard, app state, API headers, page/hash navigation.
frontend/src/components/AdminPage.tsx	Admin Console.
frontend/src/components/ChatSidePanel.tsx	Graph Copilot UX.
frontend/src/components/FlowCanvas.tsx	Graph visualization.
backend/src/index.js	Express composition/root routes/health.
backend/src/utils/helper.js	Unified DB, workspace/graph access helpers, chunk/token utilities.
backend/src/db/database.js	SQLite singleton, schema, WAL, migrations.
backend/src/db/schema.sql	20-table relational schema.
backend/src/routes/public.js	Unauthenticated published-domain API.
backend/src/routes/graphs.js	Graph catalog/CRUD and visibility.
backend/src/routes/graph.js	Nodes/edges/actors/work-items/interest scope.
backend/src/routes/auth.js	Login/register/me and isolated signup.
backend/src/routes/admin.js	Admin summary/users/import/graph editing.
backend/src/routes/workspaces.js	Workspace, projects, actor candidates, templates/cloning.
backend/src/routes/rag.js	Graph-aware document ingest/search.
backend/src/routes/copilot.js	Context builder, RAG, history/session, LLM/offline response.
backend/src/engines/llm.js	External LLM + Offline AI gateway.
offline-ai/rnn_model.py	Hybrid local retrieval/synthesis engine; legacy filename retained.
offline-ai/server.py	Zero-dependency private HTTP service.
Data/	Local knowledge corpus.
test/smoke.py	No-dependency API smoke test.
DEPLOY.md	Production topology and deployment guidance.

Конец документа. Источником истины для конкретного поведения остаётся код текущей версии 3.0.0 и ``backend/src/db/schema.sql``.